Øving 2: DNS, Epost og Web

Del 1: DNS

Det er flere siktemål med denne DNS-øvingen.

- Det første målet er å bli kjent med konfigurering på din egen PC. Du skal finne adressen til din lokale navnetjener og hva den heter. Her brukes kommandovindu på PC (ledetekst eller CMD på Windows).
- Videre skal du gjøre spesifikke DNS-oppslag hvor ett domenenavn brukes for ulike tjenester, web og epost, dette for å undersøke typefeltet i Ressursrecord.
- Til slutt skal Wireshark brukes til å finne levetiden for svar på DNS-oppslag, dvs hvor lenge de kan mellomlagres i cache.

1. **Hva heter din lokale navnetjener? Vi søker etter domenenavnet.**

Steg1: Finn hvilken/hvilke DNS-tjenere din PC er konfigurert med når du er *koplet på campusnettet*.

Start Ledetekst/CMD og gi kommando: *CMD>ipconfig /all*. Merk: Uten tilføyelsen */all* vil bare de mest sentrale parameterne vises, og ikke DNS-adresser.

SVAR: (Bruk gjerne skjermklipp)

scutil --dns (på mac)

```
➜  ~ man grep
➜  ~ scutil --dns | grep "nameserver"
  nameserver[0] : 129.241.0.200
  nameserver[1] : 129.241.0.201
  nameserver[0] : 192.168.10.1
  nameserver[0] : 129.241.0.200
  nameserver[1] : 129.241.0.201
```

Den som ender med .1 er min lokale DNS, altså ruteren hjemme (jeg kobler opp mot NTNU sitt nettverk gjennom VPN)

Steg 2: Når dette DNS-oppslaget gjøres med tilkopling til NTNU-nettet vil det vises at jeg kan velge blant *fire* ulike IP-adresser til lokale navnetjenere, to IPv4 og to IPv6. Bruk *CMD>nslookup <IP>* til å finne navnet til den første IPv4-adressen.

SVAR:

Merk: Et tjenernavn kan være registrert både med IPv4- og IPv6 adresse men allikevel være på samme maskin.

```
➜  ~ nslookup 129.241.0.200
Server:         129.241.0.200
Address:        129.241.0.200#53

Non-authoritative answer:
200.0.241.129.in-addr.arpa      name = dnscache1.ntnu.no.

Authoritative answers can be found from:

➜  ~ nslookup 129.241.0.201
Server:         129.241.0.200
Address:        129.241.0.200#53

Non-authoritative answer:
201.0.241.129.in-addr.arpa      name = dnscache2.ntnu.no.
```

2. **Finn IP-adressen til eposttjener for NTNU**

Domenenavnet til NTNU er ntnu.no. Dette navnet brukes jo blant annet for adressering i epost og web, og disse tjenestene er plassert på ulike maskiner av kapasitetsmessige årsaker. Dermed må et domenenavn kunne ha flere IP-adresser, avhengig av hvilke tjenester man bruker. Hvert ressursrecord i DNS har derfor angitt hvilken tjenestetype dette gjelder for.

Et forbehold: NTNU kjører store tjenester fordelt på flere serverer for redundans og lastbalansering. Når vi i oppgaven her søker på adresser og navn på tjenere så viser ikke dette hele bildet av NTNUs serverpark og hvordan de er knyttet sammen. Svarene kan derfor variere fra en PC til en annen.

Steg 1: Søke med nslookup på bestemt type ressursrecord (RR)

Det er 15-20 talls tjenestetyper av RR, de vanligste er

- A (IPv4)
- AAAA (IPv6)
- MX (Mail Exchange)
- NS (Name Server)
- CNAME (Alias, et navn som peker til et annet navn)
- PTR (Pointer)

Sett MX (for «Mail eXchange») og søk på ntnu.no i ledeteksten:

```
C:\Users\olav>nslookup
Default Server:  dnscache1.ntnu.no
Address:  2001:700:300::200

> set type=mx
>
```

Når man søker på ntnu.no med type MX vil man få *navnet* på en ny tjener til svar. Hva er navnet til NTNUs eposttjener?

SVAR:

```
➜  ~ nslookup
> set type=mx
> ntnu.no
Server:         129.241.0.200
Address:        129.241.0.200#53

Non-authoritative answer:
ntnu.no mail exchanger = 10 mx.ntnu.no.
```

Nå må man gjøre et nytt oppslag for å spørre etter IP-adresse for denne navngitte tjeneren. Endre til «set type= AAAA» for å finne IPv6 til eposttjener.

Hva IPv6 til NTNUs eposttjener?

SVAR:

```
> set type=AAAA
> ntnu.no
Server:         129.241.0.200
Address:        129.241.0.200#53

Non-authoritative answer:
ntnu.no has AAAA address 2001:700:300:6::102
```

3. **Finn IP-adressen og det «egentlige» navnet til tjeneren for «ntnu.no»**

Gjør et nslookup-søk med type AAAA på «ntnu.no». Kjør så et nytt søk på denne IPv6-adressen, da får man navnet på tjeneren som er knyttet til denne adressen. Merk: Her vil man finne at tjeneren har et annet navn enn «ntnu.no». Det betyr at *ntnu.no* er «merkenavnet» på Internett, men at de ulike tjenerne bak dette merkenavnet har sine egne navn.

Hva er IPv6 til ntnu.no, og hva er navnet til denne tjeneren?

SVAR:

```
➜  ~ nslookup
> set type=AAAA
> ntnu.no
Server:         129.241.0.200
Address:        129.241.0.200#53

Non-authoritative answer:
ntnu.no has AAAA address 2001:700:300:6::102

Authoritative answers can be found from:
> 2001:700:300:6::102
Server:         129.241.0.200
Address:        129.241.0.200#53

Non-authoritative answer:
2.0.1.0.0.0.0.0.0.0.0.0.0.0.0.0.6.0.0.0.0.0.0.3.0.0.0.7.0.1.0.0.2.ip6.arpa        name = lvs160vip02.it.ntnu.no.
```

Del 2 - EPOST

Hensikten med denne øvingen er å vise hvor enkelt det er å kople seg på en epost-tjener for å sende en epost, og med det vise hvordan SMTP-dialogen mellom klient og tjener fungerer.

Det skal kjøres en manuell Telnet-sesjon mot eposttjeneren på NTNU som om man selv var en ordinær epostklient. Denne sesjonen er ikke ment å fullføres, altså sende en epost, men hvis man allikevel forsøker på det så skal det uansett IKKE benyttes annen avsenderadresse enn sin egen legitime epostadresse. Det vil i så fall kunne gi meget uønskede konsekvenser for avsender.

1. Åpne en telnet-sesjon: CMD>*telnet*
2. Opprett forbindelse mot NTNUs eposttjener: *o mx.ntnu.no 25*
   (Betyr open TCP mot port 25 som SMTP lytter på)
3. Det opprettes en TCP-sesjon og Tjeneren presenterer seg med kode 220, tjenesten er klar
4. Neste steg i dialogen er at klienten skal presentere seg, men for denne testen hopper man over det og skriver bare <*mail from:xx*>
5. Tjeneren gir da en feilmelding og forteller deg hva du egentlig skulle ha sendt.

Oppgave: Hva er det SMTP svarer i linje 6?

SVAR: 503 5.5.1 Error: send HELO/EHLO first

Dette er en utmerket demonstrasjon av dialogen mellom klient og tjener, og hvor enkelt det er å kople seg på en tjener. Det kreves ingen pålogging. Tilkoplingen kan riktignok krypteres med TLS ved å kople seg på port 587, men det er bare for å ha konfidensiell forbindelse mellom klient og tjener.

I teknisk forstand er det strengt tatt ikke noe som heter «falske eposter», men vi har epost med uønsket innhold, falsk avsender osv. Derfor gjør epost-tjenere en stor jobb med å filtrere innholdet på svartelistet IP-adresser og nøkkeltekst i innholdet. For å gi en indikasjon på omfanget så får NTNU omtrent 1 million eposter daglig, og litt forsiktig anslått er ca. 80% av dette uønsket epost.

Del 3 - Webtjenesten og HTTP

Hensikten med denne øvingen å forstå hvordan HTTP fungerer, nærmere bestemt hvordan headerlinjer styrer dialogen mellom klient og tjener. Dette skal illustreres med Wireshark pakkefangst av HTTP/ 1.1. Vi kan ikke gjøre en tilsvarende undersøkelse av **HTTPS** fordi den har innhold kryptert med TLS. TLS vil bli undersøkt nærmere i neste øving.

A. Innholdet av HTTP-meldinger

I første del av øvingen benyttes det en ferdig Wireshark-pakkefangst fra et oppslag på *datakom.no* (hentes på Blackboard https://ntnu.blackboard.com/bbcswebdav/pid-2988540-dt-content-rid-89484292_1/xid-89484292_1). Studer innholdet i pakke nr 13, respons fra HTTP-tjener. Denne pakken er siste del i overføringen av hele index.html fila

Når ble denne pakkefangsten tatt?

- **Svar**: Date: Tue, 12 Jan 2021 09:31:28 GMT\r\n

Når var denne index-fila sist endret?

- Svar: Last-Modified: Wed, 23 Sep 2020 13:02:56 GMT\r\n

Hvor mange TCP-segmenter ble benyttet for å laste ned index-fila, og hvor store var de?

- **Svar**: [5 Reassembled TCP Segments (6957 bytes): #9(1460), #10(1460), #11(1460), #12(1460), #13(1117)]. 5 segmenter med størrelse til sammen 6757 bytes.

I pakke nr 13 viser Wireshark svaret fra tjener på klientens tidligerer request i linje 5 om å bruke *vedvarende forbindelse*. Formatet kalles «kolonseparert navn/verdi-par». Hvordan er denne utvekslingen?

- **Svar:**
  Klient spurte (i linje 5): Connection: Keep-Alive
- Tjener svarer: Keep-Alive: timeout=5, max=100
  Og dette svaret betyr: HTTP Keep-Alive headerfeltet sier hvordan tilkoblingen mellom klient og tjener som holder åpent basert på parameterne. Vanligvis avsluttes tilkoblingen etter hver request/response. Paramterne: Timeout er antall sekunder tilkoblingen kan være idle. Max er antall requests som kan sendes før tilkoblingen avsluttes

HTTP gjøres tilstandsfull ved bruk av cookies. Hva betyr det at en forbindelse er tilstandsfull? Hvordan brukes cookies for å oppnå dette i dialogen mellom klient og tjener?

- **Svar:** HTTP er tilstandsløs ved default. Webservereen (tjener) oppretter en unik indentifikator når man besøker en nettside. Webserveren sender denne responsen med set-cookie (serveren lager cookien). Nettleseren (klient) lagrer cookien i en lokal fil. Og ved senere requests sender nettleseren med cookien.

Det er unødvendig å laste ned uendrede filer som allerede finnes i mellomlager. Hvordan oppnår HTTP dette i dialog mellom klient og tjener?

Merk1: Dersom man sletter mellomlagrede filer så kan man kunne bruke Wireshark for å studere TCP-segmenter for nedlasting av store filer.

Merk 2: PHP-genererte html-filer vil alltid lastes ned fordi de er dynamisk genererte, alltid nye versjoner.

- **Svar:** Klienten lagrer kopier av webobjekter lokalt. Da slipper man å laste objekter på nytt hver gang da det er sløsing av båndbredde og tid. Ved neste forespørsel brukes If-modified_since headeren og serveren svarer enten med hele filen på nytt eller 304 Not Modified hvis filen er uendret.

Finn en pakke med HTTP-GET. Høyreklikk på denne pakken og velg Follow/HTTP-stream. Hva skjer?

Svar: Man får opp header request og respons og body for innholdet til HTML filen

Pakke 1 og 2 inneholder begge et TCP-SYN-flagg. Observer at ulike TCP-portnummer knyttes til disse. Hvilken effekt har dette for webtjenesten?

Svar: Begge mappes til HTTP port 80, men har ulik source port. Slikt skiller TCP samtidige forbindelser fra samme klient til samme server. Klienten oppretter altså separate forbindelser til samme webserveren (port 80). At TCP lar klienten hente ressurser parallelt fører til lavere tid brukt på nedlasting. Dette fører til overhead da hver TCP forbindelse trenger sine headers.